Project – Milestone 2

Contents

Group members 2
Member Roles 2
Chosen Programming Language 2
Communication Method 2
Project Gantt Chart 3
GitHub Link 4
Solutions 4
- Task 1 – Problem 1 Brute Force 4
  - Input 4
  - Output 4
  - Constraints 4
  - Pseudocode 5
- Task 2 – Problem 1 Greedy Solution 5
  - Input 5
  - Output 5
  - Constraints 6
  - Pseudocode 6
- Task 3 – Dynamic Programming Algorithm for Problem 1 7
  - Input 7
  - Output 7
  - Constraints 7
  - Pseudocode 7
- Task NUMBER 8
  - Input 8
  - Output 8
  - Constraints 8
  - Pseudocode 9
- Task NUMBER 10

Input 10
Output 10
Constraints 10
Pseudocode 11

Group members

Austin Hall
email: austinhall@ufl.edu
GitHub: aahall05

Stephen Kramer
email: kramers@ufl.edu
GitHub: sokramerz

Dajana Seitllari
Email: seitllarid@ufl.edu
Github: seitllarid

Richard Huang
Email: rhuang3@ufl.edu
Github: richardhuang12

Member Roles

Austin Hall – Product Manager

Stephen Kramer – Scrum Master

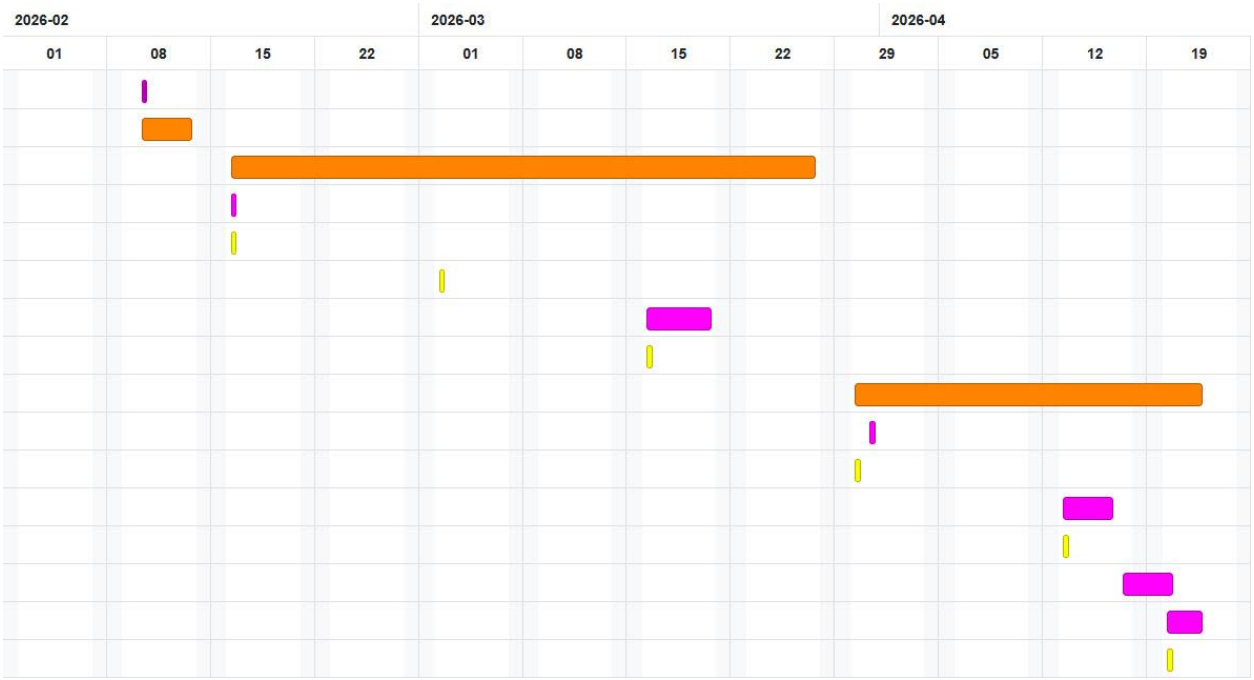
Chosen Programming Language

Our group has decided to implement our project using BLANK

Communication Method

The group created a Discord server to communicate. The discord contains our emails and phone numbers for emergency situations.

Project Gantt Chart



ID	Task Name	Start	End	Duration	Color
1	Project file and Github creation	2026-02-10	2026-02-10	1 day	
2	Milestone 1	2026-02-10	2026-02-13	4 days	
3	Milestone 2	2026-02-16	2026-03-27	30 days	
8	MS2 - Implementation planning	2026-02-16	2026-02-16	1 day	
4	Team Meeting 1	2026-02-16	2026-02-16	1 day	
5	Team Meeting 2	2026-03-02	2026-03-02	1 day	
9	MS2 - Implementation Design completed	2026-03-16	2026-03-20	5 days	
6	Team Meeting 3	2026-03-16	2026-03-16	1 day	
11	Milestone 3	2026-03-30	2026-04-22	18 days	
12	MS3 - Planning	2026-03-31	2026-03-31	1 day	
7	Team Meeting 4	2026-03-30	2026-03-30	1 day	
16	MS3 - Implementation Finished	2026-04-13	2026-04-16	4 days	
14	Team Meeting 5	2026-04-13	2026-04-13	1 day	
10	MS3 - Testing	2026-04-17	2026-04-20	2 days	
15	Project Finalization	2026-04-20	2026-04-22	3 days	
17	Team Meeting 6	2026-04-20	2026-04-20	1 day	

GitHub Link

https://github.com/aahall05/COP4533-Project-Group-6.git

Solutions

Task 1 – Problem 1 Brute Force

Input

A matrix A of dimensions $m \times n$ $(1 \leq m, n \leq 1000)$, where each element $A[i][j]$ $(0 \leq A[i][j] \leq 10^5)$ represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j_1, j_2, profit)$ where:

• i $(1 \leq i \leq m)$ is the index of the chosen stock.

• j_1 $(1 \leq j1 \leq n)$ is the day when you would buy the stock to maximize profit.

• j_2 $(1 \leq j_2 \leq n)$ is the day when you would sell the stock to maximize profit.

• Profit is the maximum profit achievable through this single transaction (sell price minus buy price)

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. You can only sell the stock you bought previously.

2. You must buy before selling; in other words, j_1 must be less than j_2.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a **brute force** algorithm with a time complexity of $O(m * n^2)$ using 1-based indexing

Pseudocode

We will have an int array variable called `result` that will store the information about the stock with the largest profit found, initialized to zeros for the base case.

```
result[stock, buy date, sell date, profit]
```

There will be three variables: `i` tracks the stock, `j1` tracks the buying day, and `j2` tracks the selling day.

```
result = [0, 0, 0, 0]
for each stock i from 1 to m
     for each buy day j1 from 1 to n - 1
          for each sell day j2 from j1 + 1 to n
               profit = A[i][j2] - A[i][j1]
               if profit > result.profit
                    result = [i, j1, j2, profit]
return result
```

The time complexity of the algorithm is O(m * n2) because the outer loop is O(m), and the inner loops are O(n), making the total complexity O(m) * O(n) * O(n) = O(m * n2)

Task 2 – Problem 1 Greedy Solution

Input

A matrix A of dimensions $m \times n$ $(1 \leq m,n \leq 1000)$, where each element $A[i][j]$ $(0 \leq A[i][j] \leq 10^5)$ represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j_1, j_2, profit)$ where:

• i $(1 \leq i \leq m)$ is the index of the chosen stock.

• j_1 $(1 \leq j1 \leq n)$ is the day when you would sell the stock to maximize profit.

• j_2 $(1 \leq j_2 \leq n)$ is the day when you would sell the stock to maximize profit.

• profit is the maximum profit achievable through this single transaction (sell price minus buy price).

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. You can only sell the stock you bought previously.

2. You must buy before selling; in other words, j_1 must be less than j_2.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a **greedy** algorithm with a time complexity of $O(m * n^2)$ using 1-based indexing

Pseudocode

For each stock, this algorithm will track the lowest price for that stock, and as we move through n, calculate the profit if we were to sell on that day.

We will have the same result variable as in task 1 that will store the information about the stock with the largest profit found, initialized to zeros for the base case.

```
result[stock, buy date, sell date, profit]
```

We will also have a variable to track the lowest price, `lowest_price`, an array to track its index, `LP_index`, a variable `i` to track the outer stock loop, and variable `j` to track the inner price loop.

```
result = [0, 0, 0, 0]
for each stock i from 1 to m
     lowest_price = A[i][1];
     LP_index = 1;
     for each sell day j from 2 to n - 1
          profit = A[i][j] - A[i][LP_index]
          if profit > result.profit
               result = [i, LP_index, j, profit]
          if A[i][j] < lowest_price
               lowest_price = A[i][j]
```

```
                LP_index = j
return result
```

Task 3 – Dynamic Programming Algorithm for Problem 1

Input

A matrix A of dimensions $m \times n$ $(1 \leq m,n \leq 1000)$, where each element $A[i][j]$ $(0 \leq A[i][j] \leq 10^5)$ represents the predicted price of the i-th stock on the j-th day. Rows represent stocks, columns represent days.

Output

Return a tuple $(i, j_1, j_2, profit)$ where:

• i $(1 \leq i \leq m)$ is the index of the chosen stock.

• j_1 $(1 \leq j1 \leq n)$ is the day when you would buy the stock to maximize profit.

• j_2 $(1 \leq j_2 \leq n)$ is the day when you would sell the stock to maximize profit.

• profit is the maximum profit achievable through this single transaction (sell price minus buy price).

Constraints

1. You can only perform one transaction, which involves buying and selling a single stock. You can only sell the stock you bought previously.

2. You must buy before selling; in other words, j_1 must be less than j_2.

3. The prices are non-negative integers and represent the stock's value at a specific time.

4. You want to maximize profit, so you're looking for the highest possible difference between the sell price and the buy price.

5. If no profitable transaction is possible, return a tuple (0, 0, 0, 0) to indicate this.

6. This will be a **dynamic programming** algorithm with a time complexity of O(m * n) using 1 based indexing.

Pseudocode

We'll use a result array to store the best stock, buy day, sell day, and profit found.

result = [0,0,0,0]

three variables: i tracks the stock, j tracks the day, and min_day tracks the day with lowest price seen

```
for each stock i from          1 to m
        min_day = 1
        for each day j from 2 to n
                profit = A[i][j] – A[i][min_day]
                if profit > result.profit
                        result = [i , min_day, j, profit]
                if A[i][j] < A[i][min_day]
                min_day = j
return result
```

2.5 Task – 6: Dynamic Programming Algorithm for Problem 3

Input

- Matrix A: A matrix of dimensions m x n, where each element $A[i][j]$ represents the predicted price of the $i-th$ stock on the $j-th$ day.
- Integer c: The cooldown period needed after selling a stock before another purchase can be made $(1 \leq c \leq n-2)$.

Output

Sequence of tuples (i, j_1, j_2) representing the transactions:

- i: The index of the chosen stock $(1 \leq i \leq m)$.
- j_1: The day the stock was purchased.
- j_2: The day the stock was sold to maximize profit.
- The goal is to maximize the sum of profits across all transactions while adhering to the cooldown constraint.

Constraints

1. Multiple Transactions: You can perform multiple transactions but can only hold one stock at any given time.
2. Cooldown Requirement: If you sell a stock on day j, you can't buy any stock until day $j + c + 1$.
3. Order of Operations: You have to buy before selling; therefore, j_1 must be strictly less than j_2.

4. No Profitable Trades: If no profit is possible, return an empty sequence.
5. Complexity: This algorithm uses a Top-Down Recursive approach with memorization to achieve a time complexity of $O(m * n^2)$ using 1-based indexing.

Pseudocode

```
Algorithm solveMaxProfit(priceMatrix, stockCount, totalDays,
cooldownPeriod):
    // memoMap stores [maxProfit, transactionList] for each
starting day
    memoMap = array of size (totalDays + 1), initialized to
null

    Function computeBestStartingAt(currentDay):
        // Base Case: Cannot trade if we are past the last day
        if currentDay >= totalDays:
            return (0, [])

        // Check if we have already solved this sub-problem
        if memoMap[currentDay] is not null:
            return memoMap[currentDay]

        // CHOICE A: Don't buy anything on currentDay
        // We just move to the next day to see if a better deal
starts there
        (maxProfit, bestTxns) =
computeBestStartingAt(currentDay + 1)

        // CHOICE B: Buy a stock on currentDay and sell it on
some future day 'k'
        for k from (currentDay + 1) to totalDays:
            for i from 1 to stockCount:

                // Calculate profit for this specific single
trade
                tradeProfit = priceMatrix[i][k] -
priceMatrix[i][currentDay]

                if tradeProfit > 0:
                    // After selling on day k, we jump ahead by
(1 + cooldownPeriod + k)
                    // This satisfies the "wait c days" rule
                    nextAvailableDay = 1 + cooldownPeriod + k

                    (futureProfit, futureTxns) =
computeBestStartingAt(nextAvailableDay)
                    currentTotal = futureProfit + tradeProfit
```

```
                    // If this path is better, update our best
result for this day
                    if currentTotal > maxProfit:
                        maxProfit = currentTotal
                        // Reordered tuple: (Buy Day, Sell Day,
Stock Index)
                        bestTxns = [(currentDay, k, i)] +
futureTxns

        // Cache the result before returning
        memoMap[currentDay] = (maxProfit, bestTxns)
        return memoMap[currentDay]

    // Kick off the recursion starting at Day 1
    (finalProfit, finalSequence) = computeBestStartingAt(1)
    return finalSequence
```

Complexity Analysis

- Time Complexity: $O(m * n^2)$. There are n unique states (days) to solve. For each state, we iterate through m stocks and check up to n possible sell days.
- Space Complexity: $O(n)$. The space is required for the memorization table and the recursion stack, both of which are proportional to the number of days n.

Task 4 – Dynamic Programming Algorithm for Problem 2

Input

A matrix A with a size m x n where A[i][j] represents the price of stock i on day j and integer k representing the maximum number of transactions that can be performed.

Output

Return a list of transactions (i , j1, j2) where:

- i represents the chosen stock
- j1 is the day you buy the stock
- j2 is the day you sell the stock

Constraints

1. You can perform no more than k transactions
2. The prices are non-negative because it represents money
3. The goal is to get the maximum number of total profits

4. If there is no profit, then you must return an empty list
5. You must buy before selling (j1 < j2)
6. It is a dynamic programming algorithm with a time complexity O(m * $n^2 * k$) using 1 base indexing

Pseudocode

The algorithm is going to iterate through each stock and check different buy and sell days, keeping track of the transactions it has to use to make sure we don't exceed the number of transactions allowed.

```
result = empty list

for each stock i from 1 to m

        transactions_used = 0

        for each day j from 2 to n

                if transactions_used == k

                        break

                best_profit = 0

                buy_day = 0

                for each previous day p from 1 to j-1

                        profit = A[i] [j] – A[i][p]

                        if profit > best_profit

                                best_profit = profit

                                buy_day = p

                if best_profit > 0

                        add( i, buy_day, j) to result

                        transactions_used = transactions_used + 1

return result
```

Task 7

Input

A matrix A of dimensions m × n (1 ≤ m,n ≤ 1000), where each element A[i][j] (0 ≤ A[i][j] ≤ 10^5) represents the predicted price of the i-th stock on the j-th day.

An integer c (1 ≤ c ≤ n − 2) representing the cooldown period after selling.

Output

Sequence of tuples (i, j_1, j_2) representing the transactions:

- i: The index of the chosen stock $(1 \leq i \leq m)$.
- j_1: The day the stock was purchased.
- j_2: The day the stock was sold to maximize profit.
- The goal is to maximize the sum of profits across all transactions while adhering to the cooldown constraint.

Constraints

6. Multiple Transactions: You can perform multiple transactions but can only hold one stock at any given time.
7. Cooldown Requirement: If you sell a stock on day j, you can't buy any stock until day $j + c + 1$.
8. Order of Operations: You have to buy before selling; therefore, j_1 must be strictly less than j_2.
9. No Profitable Trades: If no profit is possible, return an empty sequence.
10. Complexity: This algorithm uses a Bottom-Up Recursive approach with memorization to achieve a time complexity of $O(m * n)$ using 1-based indexing.

Pseudocode

```
Algorithm solveMaxProfit(priceMatrix, stockCount, totalDays,
cooldownPeriod):
    // dp[j] stores [maxProfit up to day j]
    dp = array of size (totalDays + 1), initialized to 0
    // txnTrack[j] stores the transaction sequence that gives dp[j]
    txnTrack = array of size (totalDays + 1), each initialized to empty list
    // Process each stock independently
    for i from 1 to stockCount:
        // Track best effective buy:
        // (price - previous profit before cooldown)
        best_buy = priceMatrix[i][1]
        best_buy_day = 1
        for j from 2 to totalDays:
            // Update best_buy using cooldown constraint
            if j - cooldownPeriod - 1 ≥ 1:
                candidate = priceMatrix[i][j] - dp[j - cooldownPeriod - 1]
            else:
                candidate = priceMatrix[i][j]
            if candidate < best_buy:
                best_buy = candidate
                best_buy_day = j
            // Compute profit if we sell today
            profit = priceMatrix[i][j] - best_buy
            // Option 1: skip today
            if dp[j - 1] ≥ dp[j]:
                dp[j] = dp[j - 1]
                txnTrack[j] = txnTrack[j - 1]
                 // Option 2: sell today
            if profit > 0:
                if j - cooldownPeriod - 1 ≥ 1:
                    prevTxns = txnTrack[j - cooldownPeriod - 1]
                else:
                    prevTxns = []
```

```
                if j - cooldownPeriod - 1 ≥ 1:
                    newProfit = profit + dp[j - cooldownPeriod - 1]
                else:
                    newProfit = profit
                if newProfit > dp[j]:
                    dp[j] = newProfit
                    txnTrack[j] = prevTxns + [(i, best_buy_day, j)]
    return txnTrack[totalDays]
```

Complexity Analysis

- Time Complexity: $O(m * n)$. There are n unique states (days) to solve. For each day, we iterate through m stocks and evaluate the highest potential profit if we were to sell on that day. We use the best_buy variable to keep track of the lowest effective cost of buying each stock so far (and its corresponding day using the best_buy_day variable). This value is adjusted by taking the stock's price and subtracting the maximum profit achievable up to the last valid day before the cooldown period, ensuring that only allowable prior transactions are considered.
- Space Complexity: $O(n)$. We use a DP array of size n to store the maximum profit up to each day, along with transaction tracking arrays of the same size.